Moderne Datenanalyse mit R: Ein Einstieg für Aktuare

Modul I: Reproduzierbares Programmieren

Seminarreihe der ÖFdV
Dietmar Hareter, Fabian Pribahsnik

Wir starten in Kürze

Dietmar Hareter

Fabian Pribahsnik

Modulübersicht

Modul I	Modul II	Modul III	Modul IV	Modul V
Reproduzierbares Programmieren	Grundkonzepte der Programmierung/ Datenaufbereitung 1	Grundkonzepte der Programmierung/ Datenaufbereitung 2	Visualisierung	Automatisierung
08.04.2026	15.04.2026	22.04.2026	29.04.2026	06.05.2026

Agenda Modul I | Die Toolbox

Teil 1 – Code versionieren

- Ein alltägliches Problem
- Versionsverwaltung: Git, SVN, GitHub, Bitbucket, GitLab
- Grundsätze der Versionsverwaltung
- Beispiel eines Workflows
- Nächste Schritte

Teil 2 – Reproduzierbare Berichte

- Jupyter
- R Markdown
- Quarto

Menti

Sind Sie mit den Konzepten von R Markdown / Jupyter vertraut?

12 Nein

5 Teilweise

1 Ja

Menti

Vorkenntnisse & aktuelle Arbeitsweise

Wie schätzen Sie Ihre Kenntnisse beim Programmieren mit R (oder vergleichbaren Sprachen wie z.B. Python) ein?

2.3

Wie hoch ist der Anteil Ihrer Projekte, bei denen Sie bereits ein Versionierungssystem (Git, SVN, etc.) nutzen?

1.5

Wie kritisch bewerten Sie die Notwendigkeit, Ihre aktuellen Analysen vollständig reproduzierbar und revisionssicher zu gestalten?

3.4

Gering

Hoch

Ein alltägliches Problem

Sie als Aktuar…

analyse.R

analyse_sent.R

analyse_v2.R

analyse_v2_final.R

Ein alltägliches Problem

analyse.R

analyse_sent.R

analyse_v2.R

analyse_v2_final.R

Ein alltägliches Problem | Versionsverlauf

analyse.R

analyse.R

analyse.R

“commit”

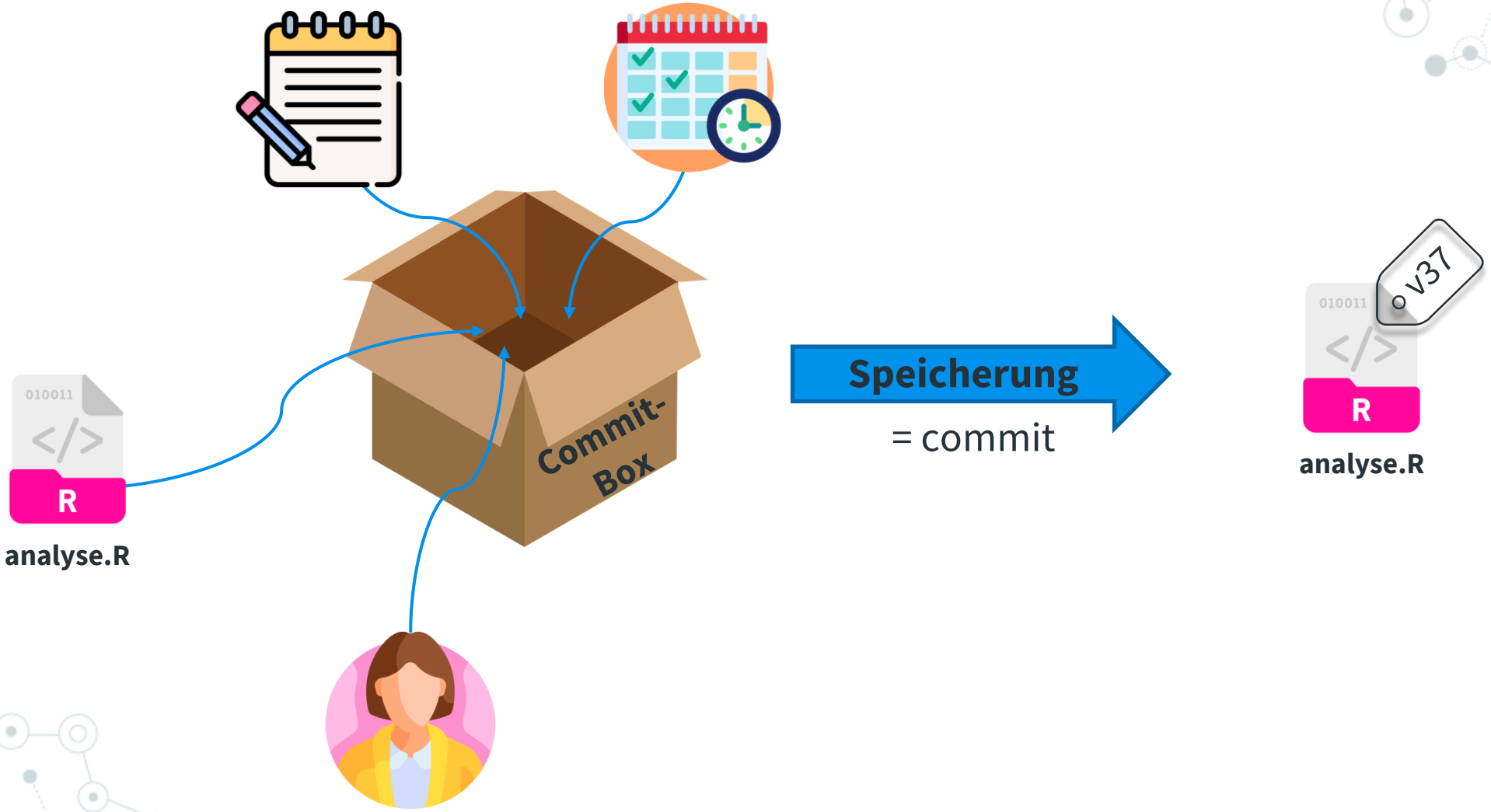
“commit”

```
10  10
11  11      # Fit a linear model
12  12      m1 <- lm(Outstate ~ Expend, data=College)
13  13
14      -   plot(College$Expend, College$Outstate)
    14  +   plot(College$Expend, College$Outstate,
    15  +        xlab = "Instructional expenditure per student",
    16  +        ylab = "Out-of-state tuition")
15  17      abline(m1, col = "red", lwd = 3)
```

```
10  11
11  12      # Fit a linear model
12  13      m1 <- lm(Outstate ~ Expend, data=College)
13  14
    15  +   # Create simple scatter plot with labels
14  16      plot(College$Expend, College$Outstate,
15  17           xlab = "Instructional expenditure per student",
16  18           ylab = "Out-of-state tuition")
    ...
```

Source: [GitHub-Repo]

Versionsverwaltung



Versionsverwaltung | Softwarelösungen

- Versionierungssoftware: Ein smartes Tool für Zusammenarbeit
- Speichert verschiedene Versionen von Dateien und ermöglicht historische Vergleich
- Funktioniert auf Ordnerebene: Zeitmaschine für alles innerhalb eines Ordners

- Host-Service für Git Projekte (Repositories)
- Bietet eine grafische Oberfläche
- Ermöglicht einfache Zusammenarbeit über Internet

Versionsverwaltung | git

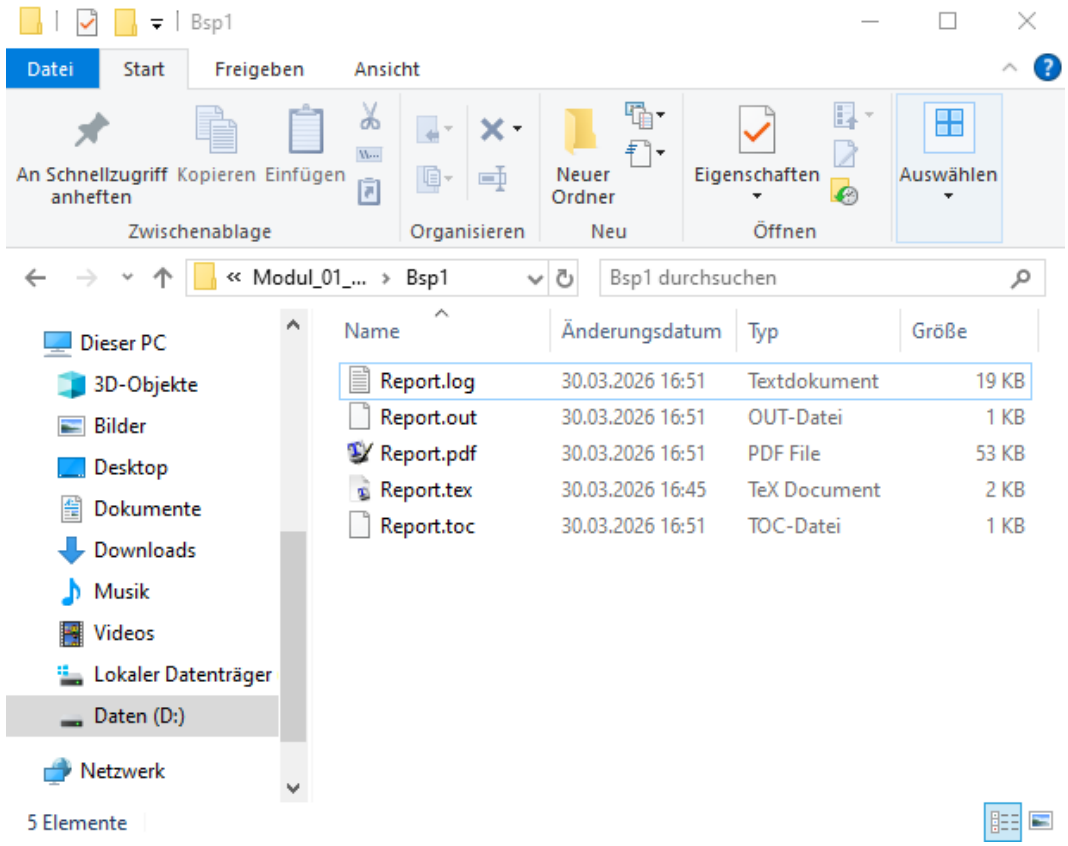
- **Installation:** Muss lokal am Computer installiert sein
- **CLI:** git ist ein CLI (command line interface) ohne grafische Oberfläche → Interaktion via Kommandozeile
- **Open Source:** Keine Lizenzgebühren, herstellerunabhängig und zukunftssicher
- **Projektbasiert:** Anwendbar auf Verzeichnisse/Order (Repository)
- **Add-Ins:** Integration in vielen Softwarelösungen möglich
  - RStudio, VS Code, JupyterLab, PyCharm, …
- **TortoiseGit:** Praktische grafische Oberfläche im Kontextmenü

Versionsverwaltung | GitHub Beispiel

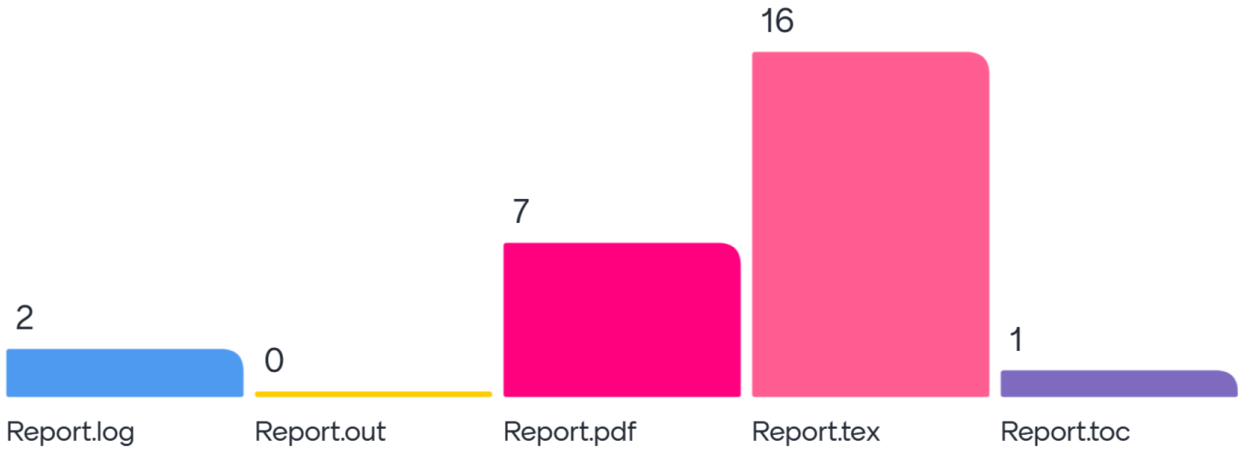
https://github.com/smalldatascience/Beispiele

https://github.com/smalldatascience/Beispiele/compare/7425651...3e79c74

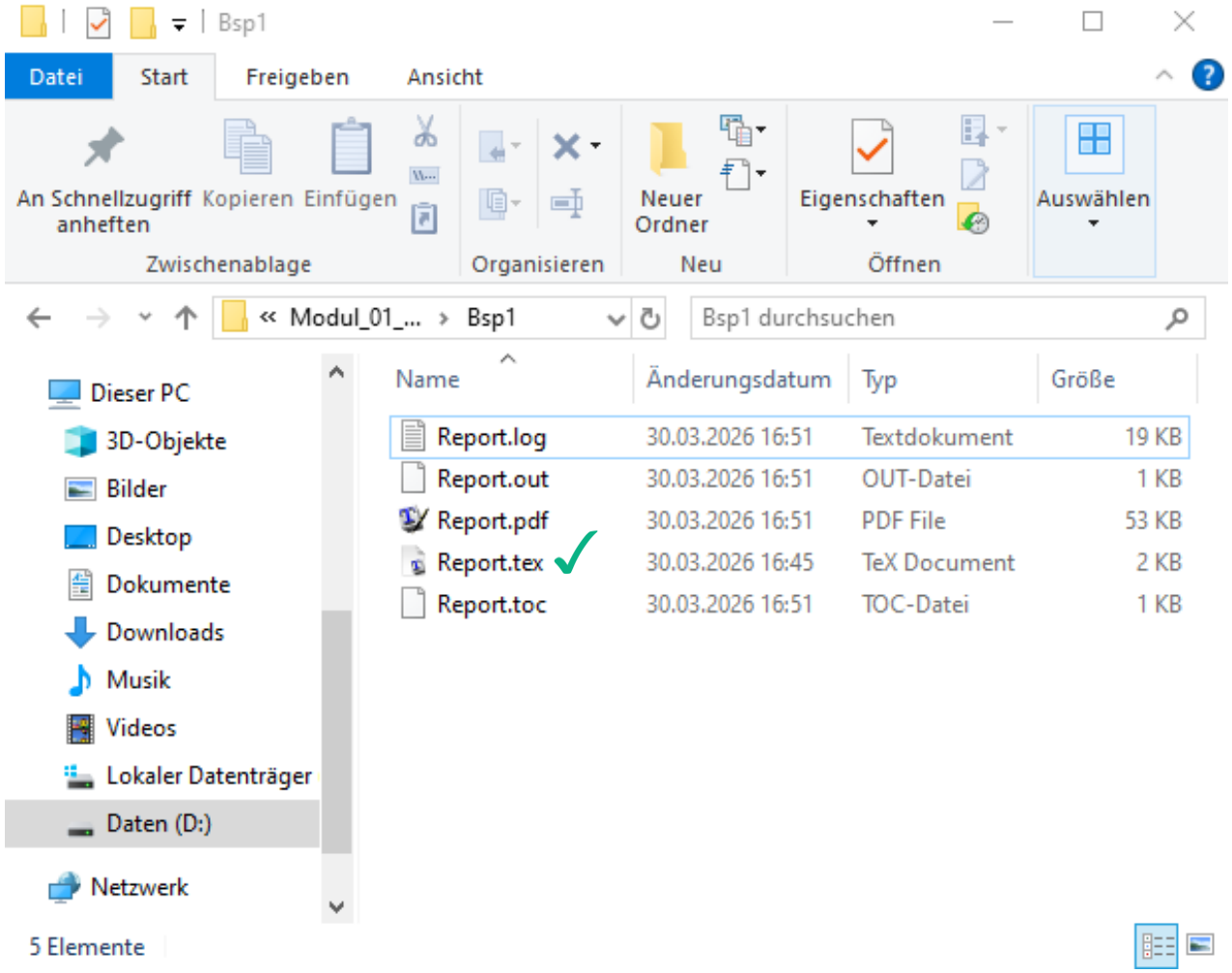
Grundsätze der Versionsverwaltung | Beispiel 1



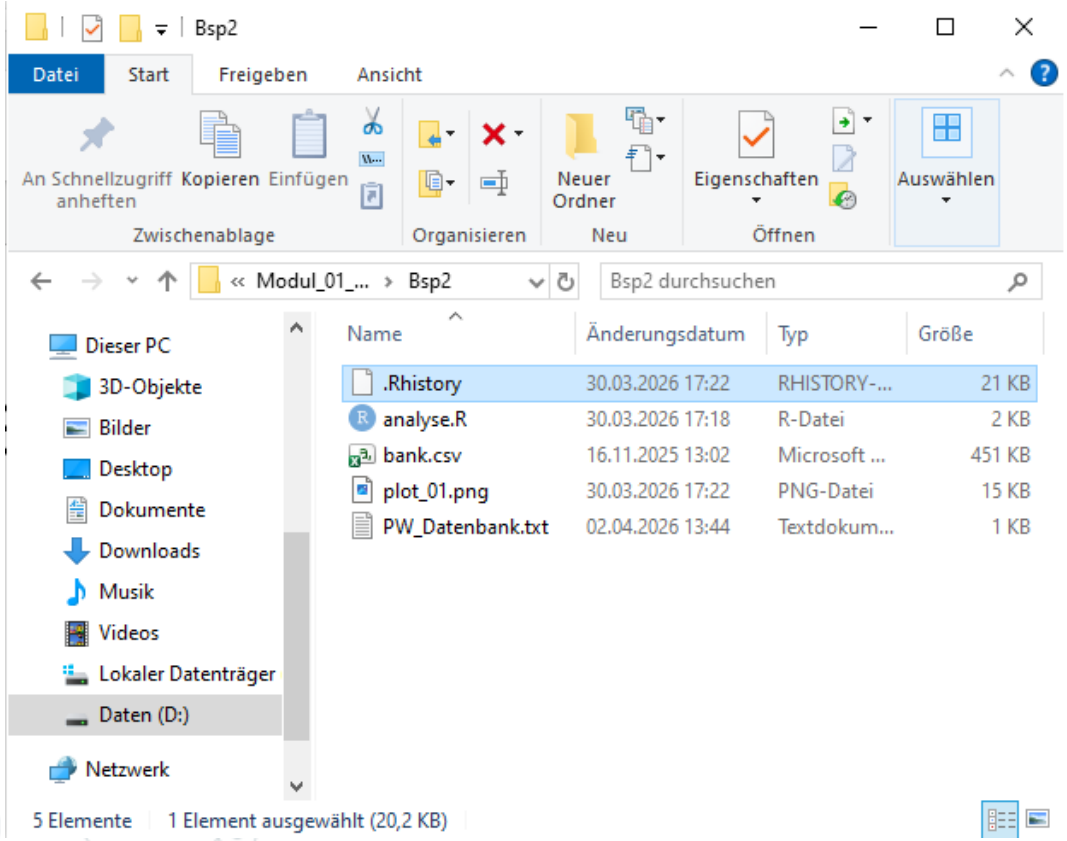
Welche Dateien würden Sie versionieren?



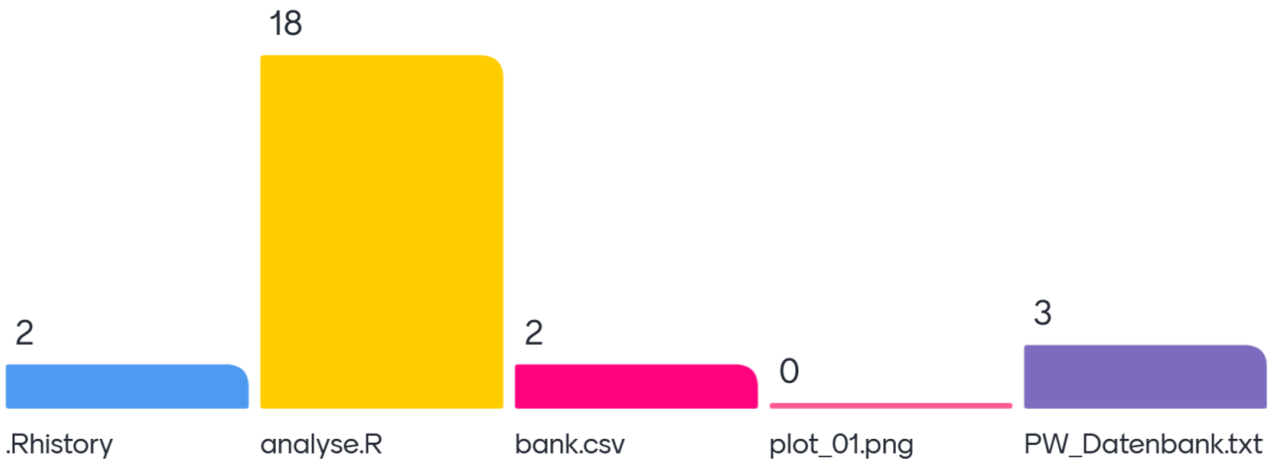
Grundsätze der Versionsverwaltung | Beispiel 1



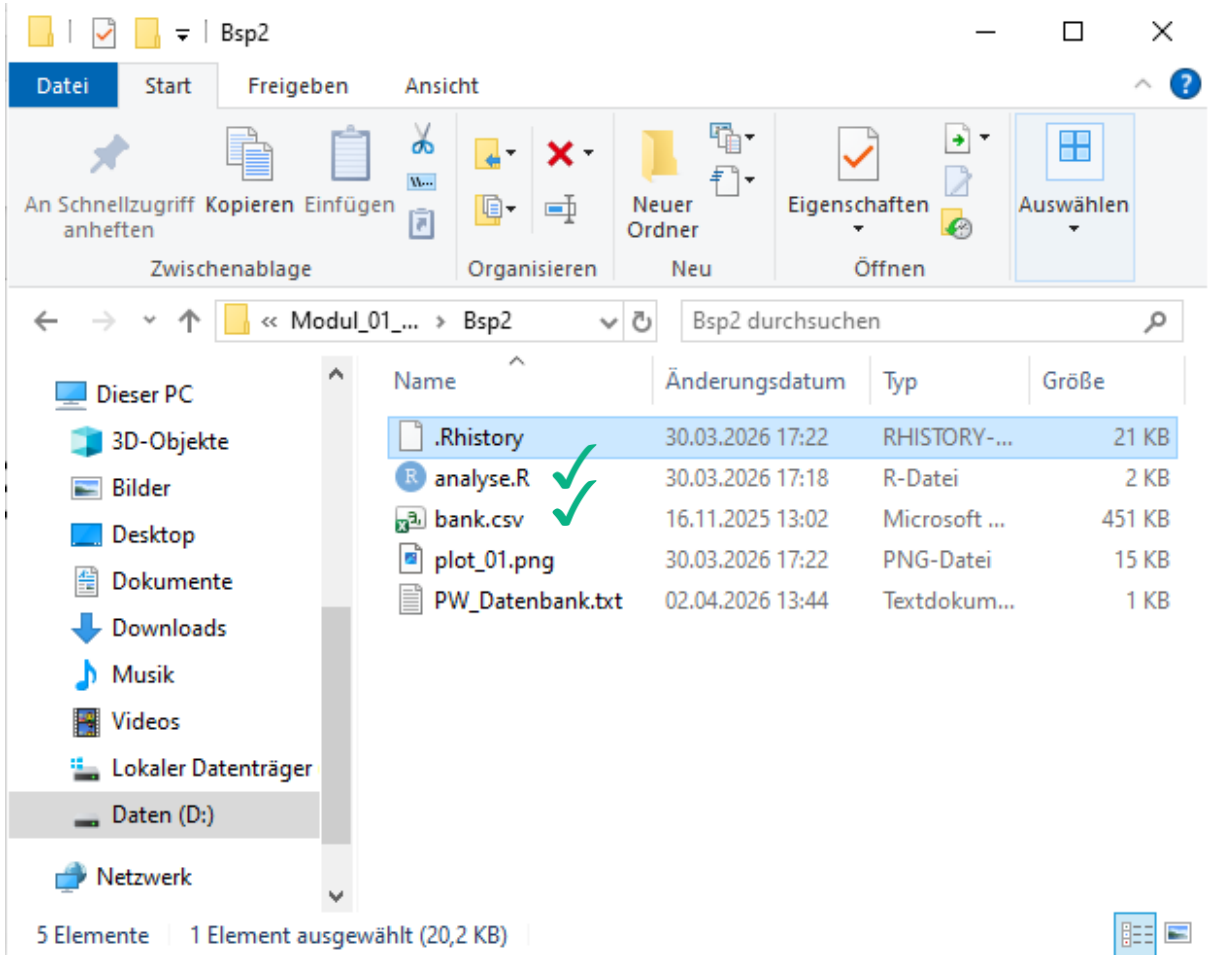
Grundsätze der Versionsverwaltung | Beispiel 2



Welche Dateien würden Sie versonieren?



Grundsätze der Versionsverwaltung | Beispiel 2



Grundsätze der Versionsverwaltung | Was wird versioniert?

- **Fast alles:** Alles, was für das Projekt relevant ist um den Code kompilieren zu können
- **Keine vertrauliche Info:** Niemals Passwörter, Zugangsdaten, … versionieren
- **Keine Output-Dateien:** Kompilierte Berichte, Grafiken, … die mit Hilfe des Codes generiert werden können
- **Keine temporären Dateien:** Daten die nutzerspezifisch sind wie bspw. .Rhistory, .RData, .Rproj.user
- **.gitignore:** Definiert welche Dateien ignorieren soll

Grundsätze der Versionsverwaltung | .gitignore

Hallo Fabian

Womit fangen wir an?

Erstelle mir eine .gitignore Datei für die Arbeit die alle .pdf und .png ignoriert. Außerdem sollen .RData .Rhistory nicht versioniert werden.

+ Tools Thinking-Modus

Gedankengang anzeigen

Hier ist der Inhalt für deine `.gitignore` Datei. Kopiere diesen Text einfach in eine Datei mit dem Namen `.gitignore` in deinem Projektverzeichnis.

Plaintext

```
# Dokumente und Bilder ignorieren
*.pdf
*.png

# R-spezifische Dateien (Temporäre Daten und Verlauf)
.RData
.Rhistory

# Optional, aber empfohlen für R-Projekte:
.Rproj.user/
```

Grundsätze der Versionsverwaltung | commits

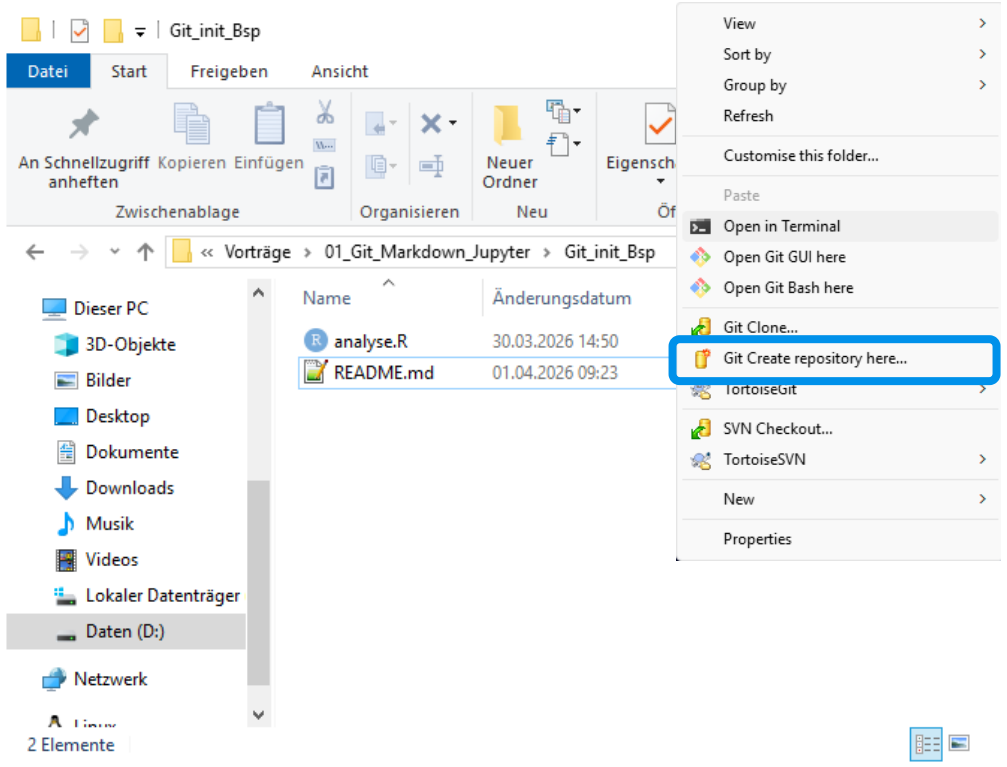
- **Atomic Commits:** Zwingt in funktionalen Einheiten zu denken, nicht in Arbeitsstunden.
- **Lauffähigen Zustand:** Jeder commit ist in sich abgeschlossen und hinterlässt den Code in einem lauffähigen Zustand.
- **Präzise Fehlersuche:** Je kleiner die commits desto besser lassen sich Fehler eingrenzen.
- **Einfacheres Zusammenführen:** Kleine Änderungen erleichtern das Lösen von Konfliktfällen (2 Kollegen ändern dieselbe Zeile)
- **Sicheres Rollback:** Fehlerhafter Code kann leicht zurückgenommen werden,

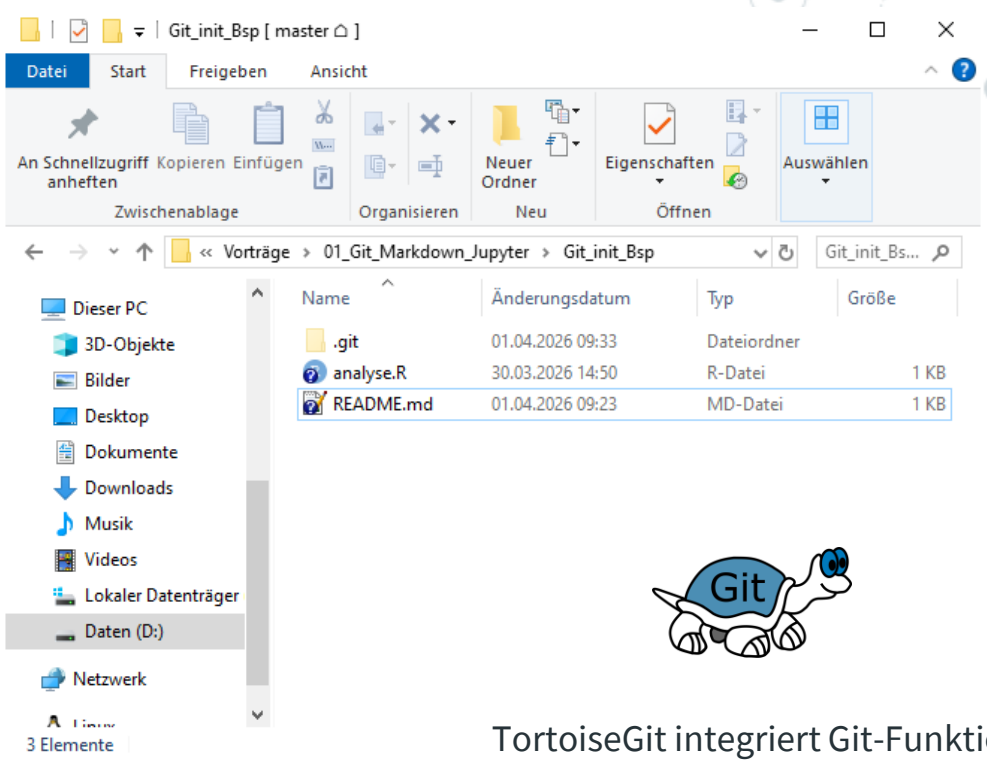
Grundsätze der Versionsverwaltung | Arbeitsablauf

1. Repository anlegen / initialisieren
2. Änderungen an Dateien vornehmen
3. Gewünschte Dateien zur Staging area hinzufügen
4. Änderungen mit Log Nachricht commiten
5. Wiederhole Punkte 2 – 4

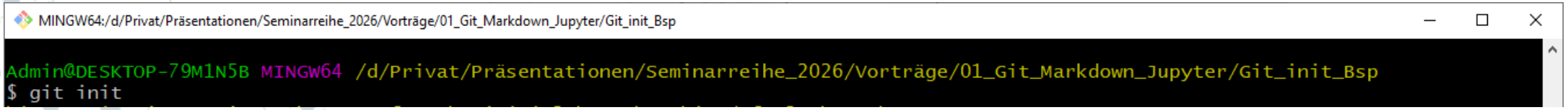
- **Repository (Repo):** Die Projektdatenbank (lokal oder remote)
- **Working directory:** Der lokaler Ordner, in dem Änderungen am Code vorgenommen werden.
- **Staging area:** Eine Liste von Dateien welche im nächsten Commit berücksichtig werden sollen.
- **Commit:** Ein lauffähiger Speicherpunkt des Projekts.
- **Log Nachricht:** Eine Notiz, die erklärt, was in einem commit geändert wurde.

Praxisbeispiel | Ein neues Repo anlegen



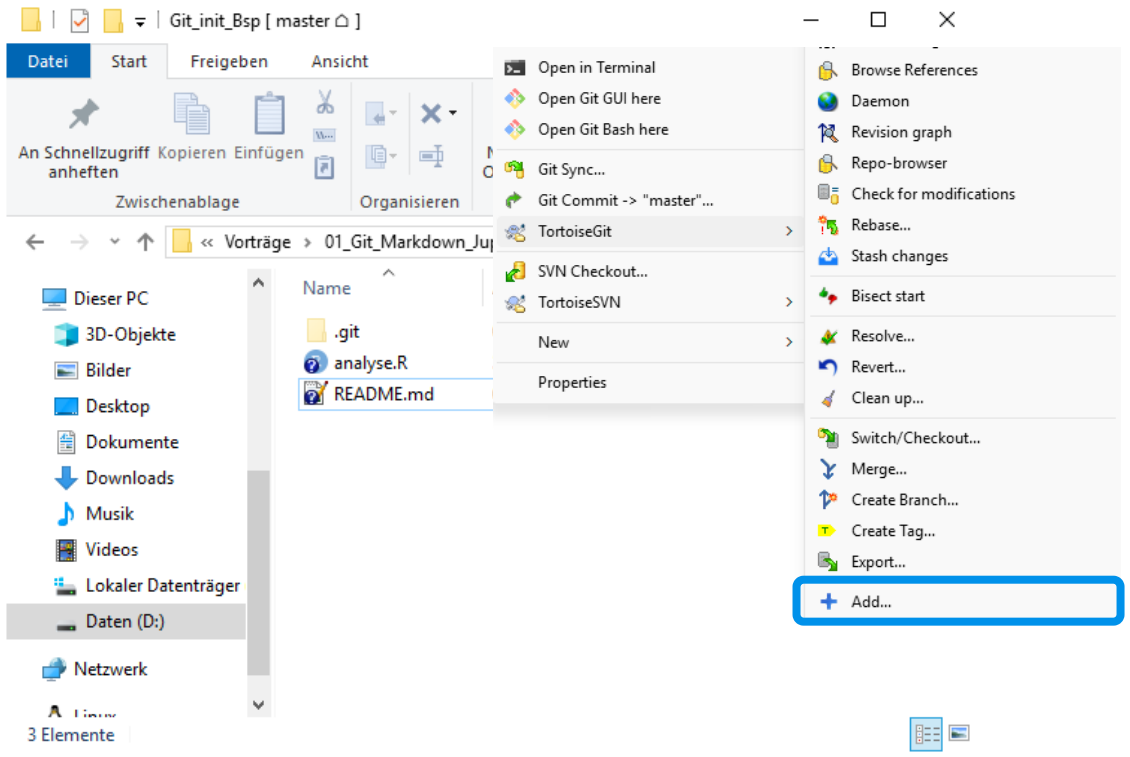


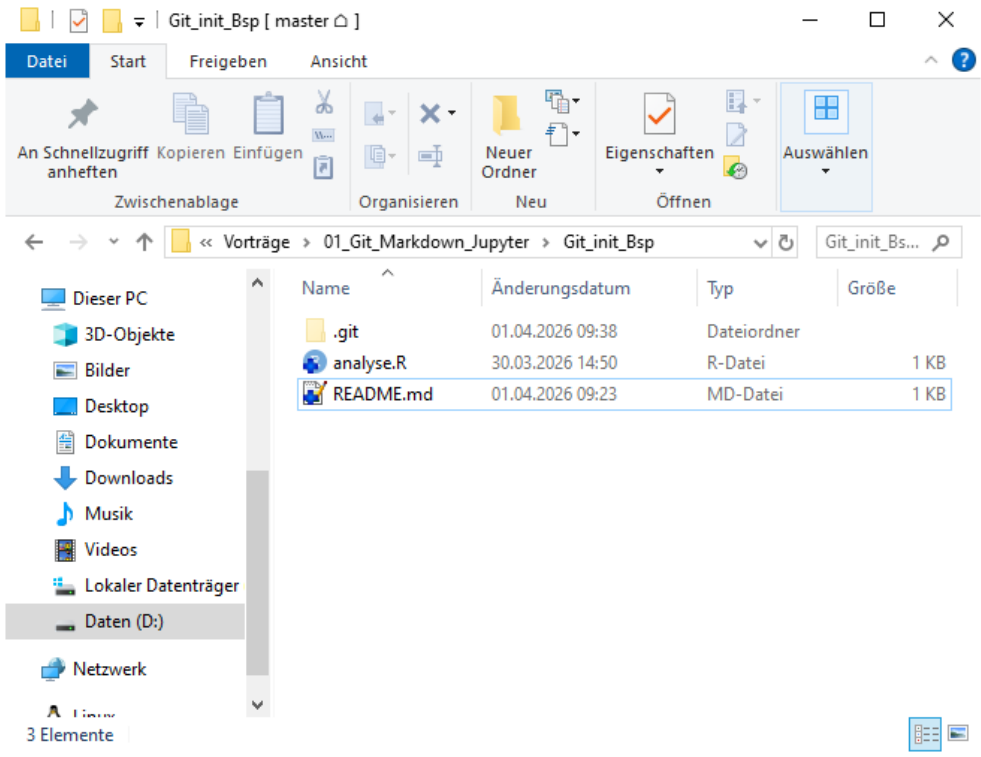

```
git init
```



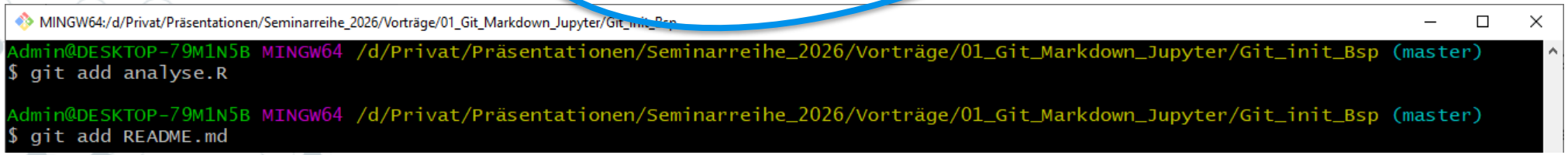
TortoiseGit integriert Git-Funktionen direkt in das Rechtsklick-Menü deines Windows Explorers und zeigt den Status via Icons an.

Praxisbeispiel | Staging area befüllen

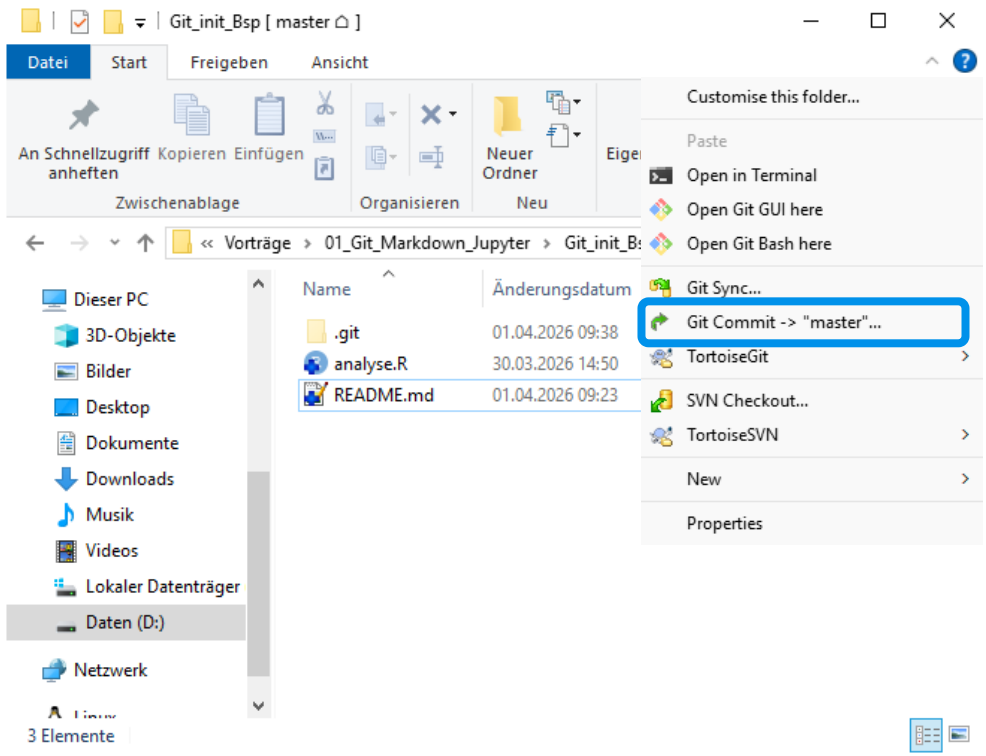


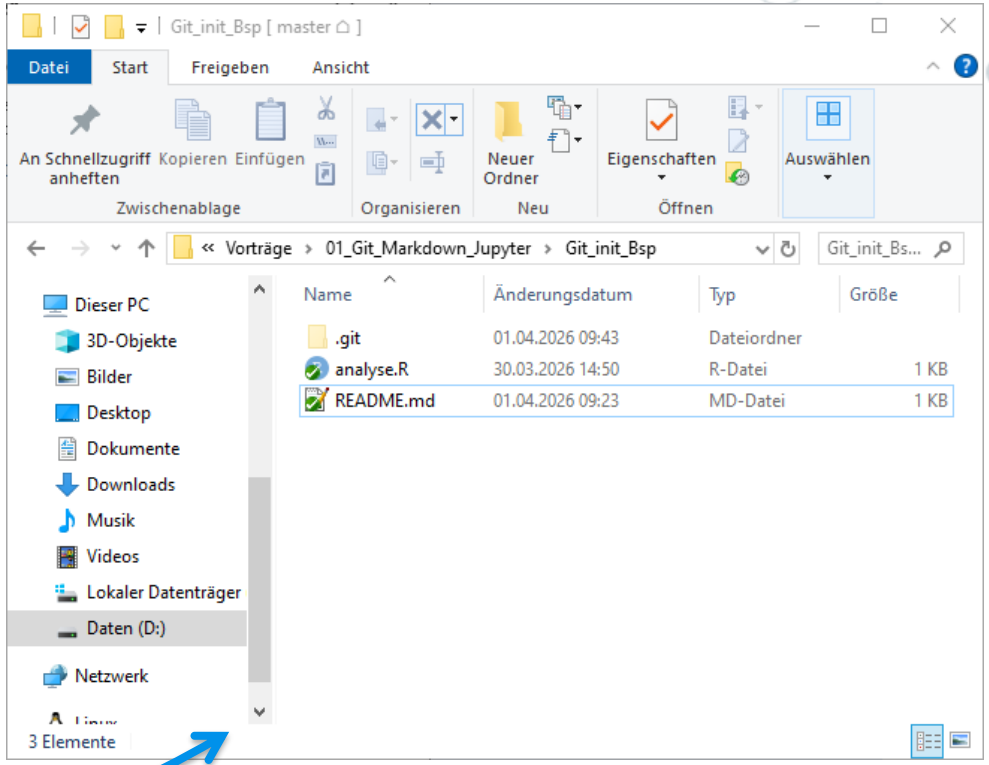


git add

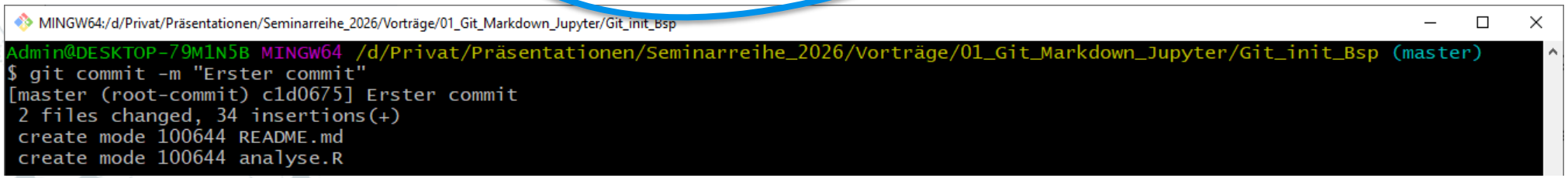


Praxisbeispiel | Commit durchführen

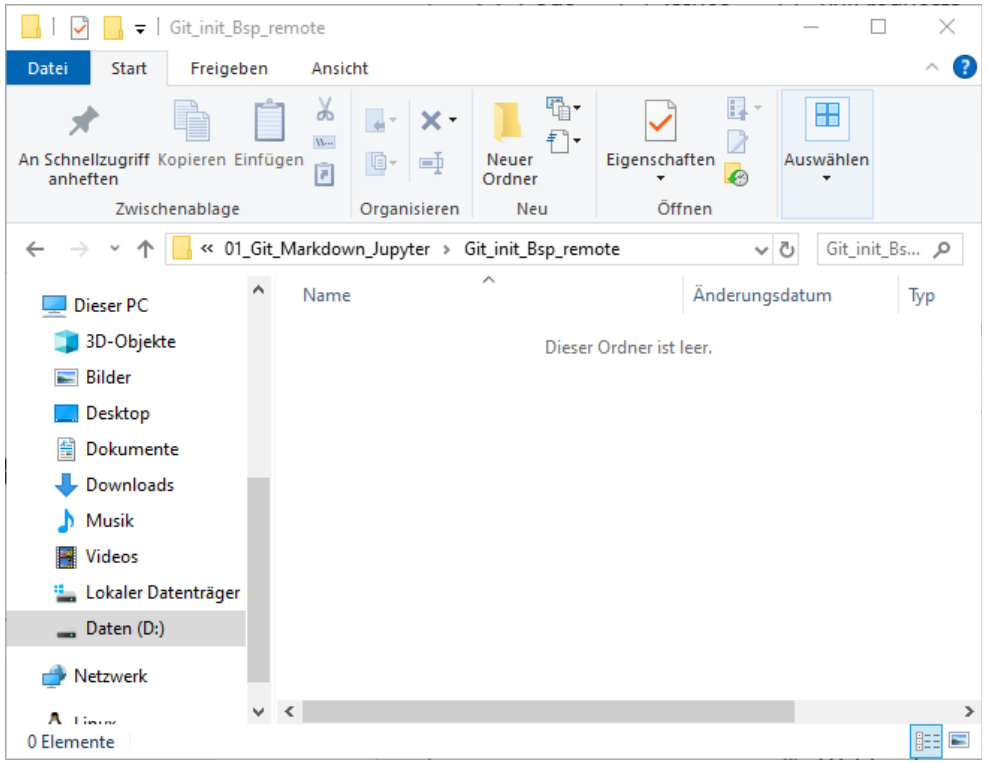


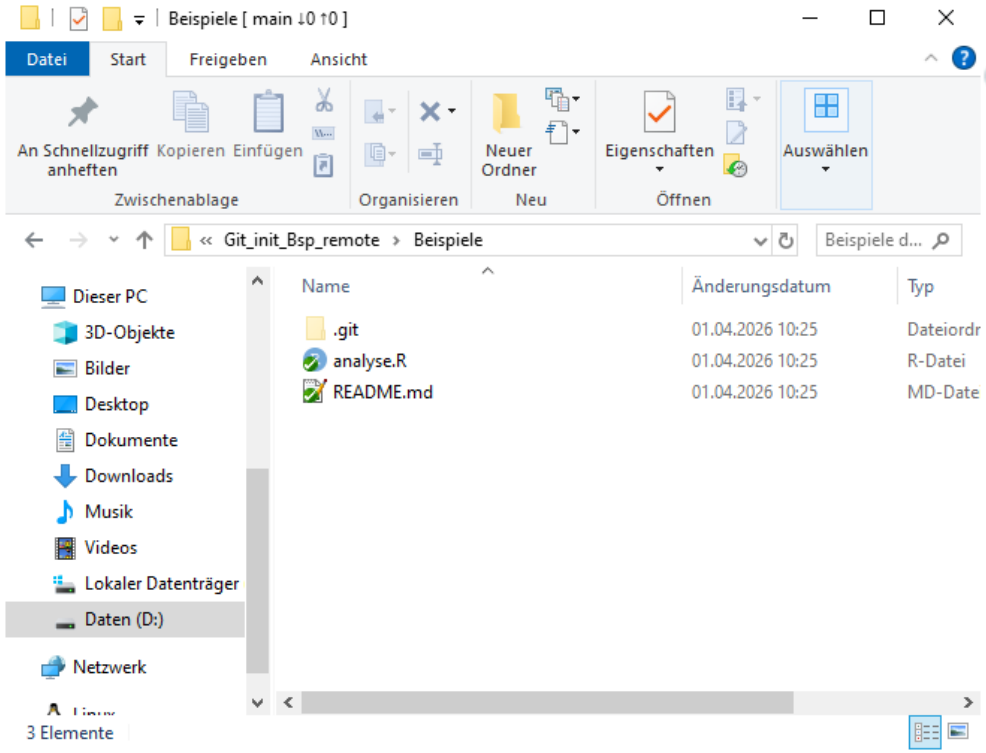



```
git commit
```

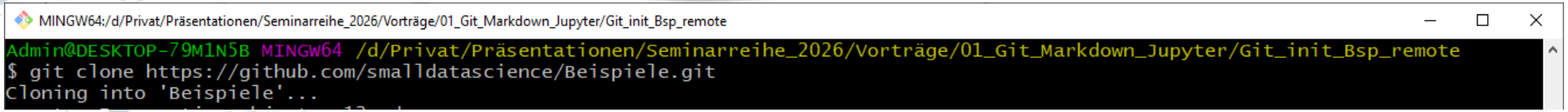


Praxisbeispiel | Ein vorhandenes Repo klonen

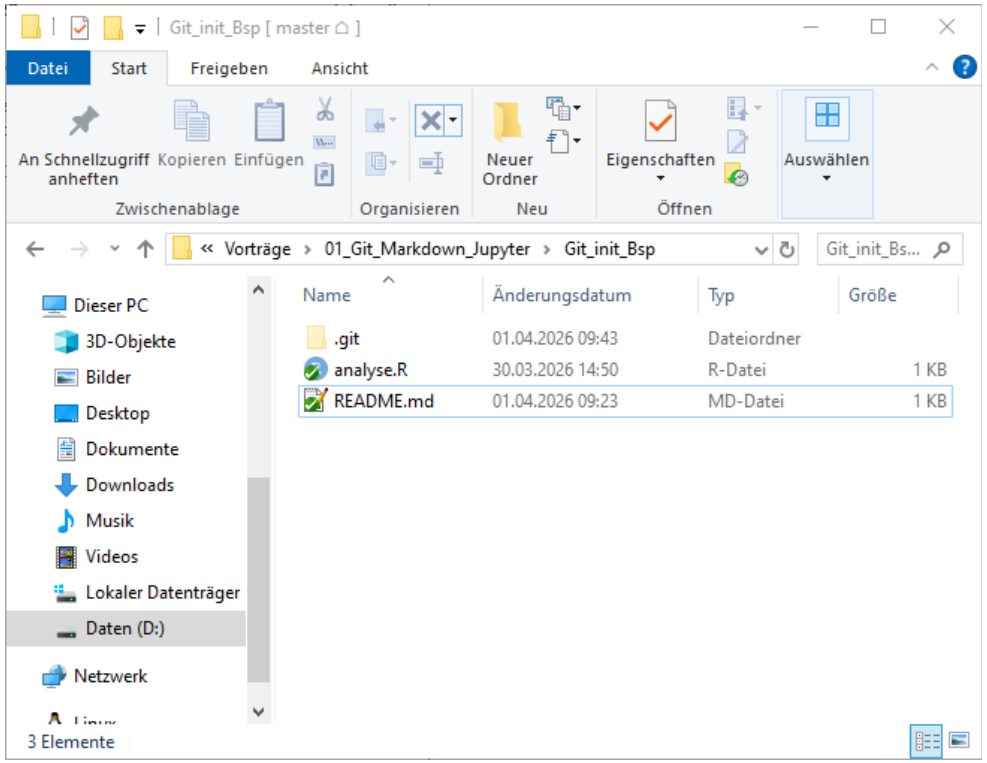


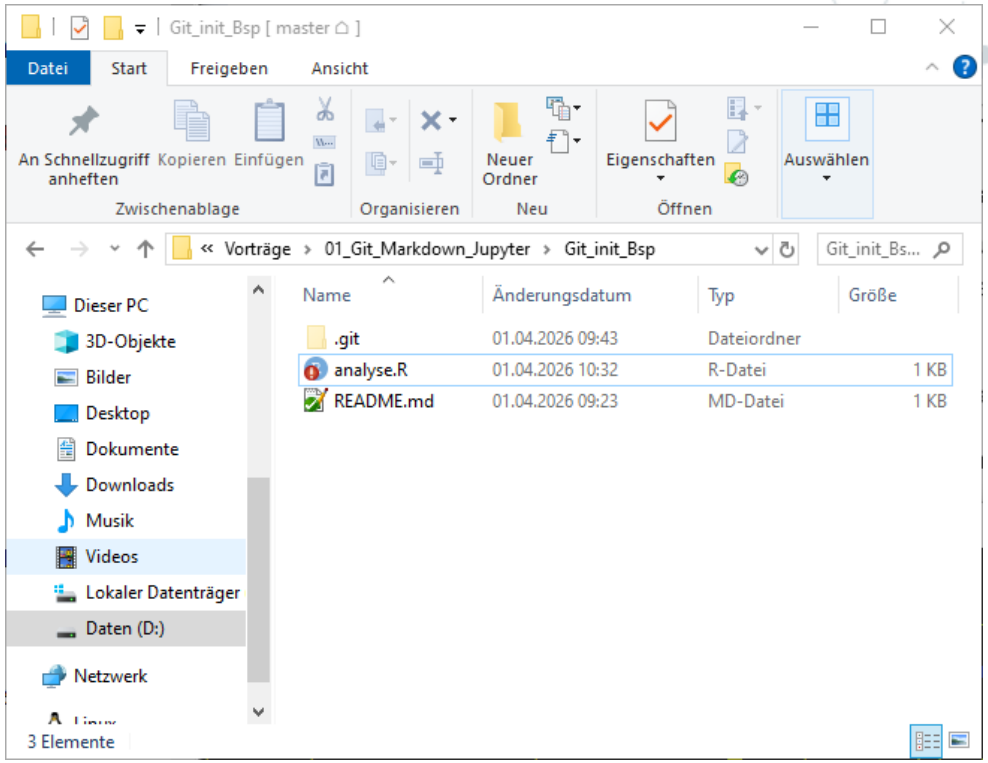



```
git clone <link>
```



Praxisbeispiel | Änderungen durchführen

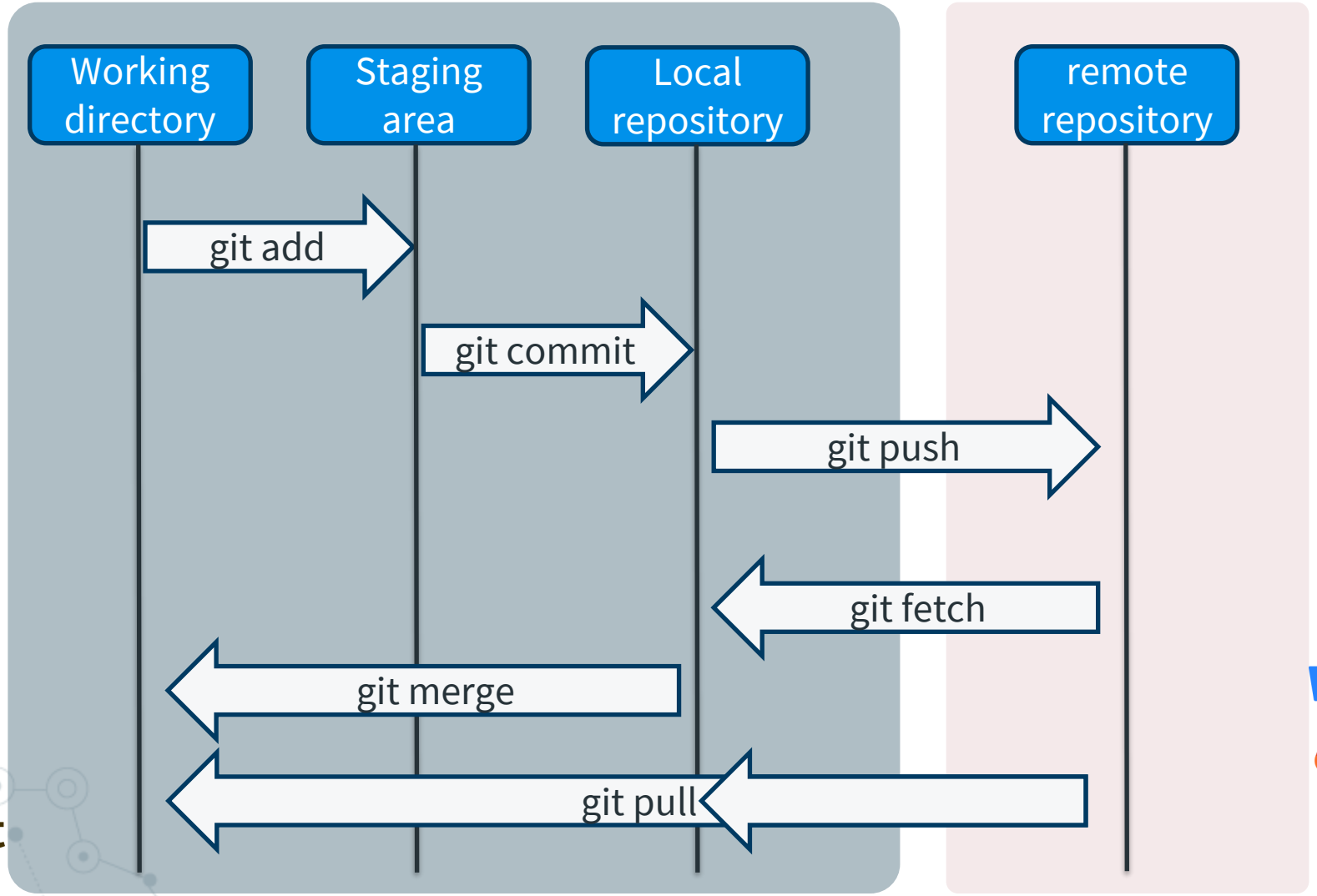





```
git add
git commit -m "Log-Nachricht"
git push (falls remote repo)
```

…

Praxisbeispiel | Verallgemeinerter Arbeitsablauf



Bitbucket
GitLab
GitHub

Nächste Schritte | Nützliche Befehle – `git diff`

`git diff` Zeigt Unterschiede in Dateien

MINGW64:/d/Privat/Präsentationen/Seminarreihe_2026/Vorträge/01_Git_Markdown_Jupyter/Git_init_Bsp

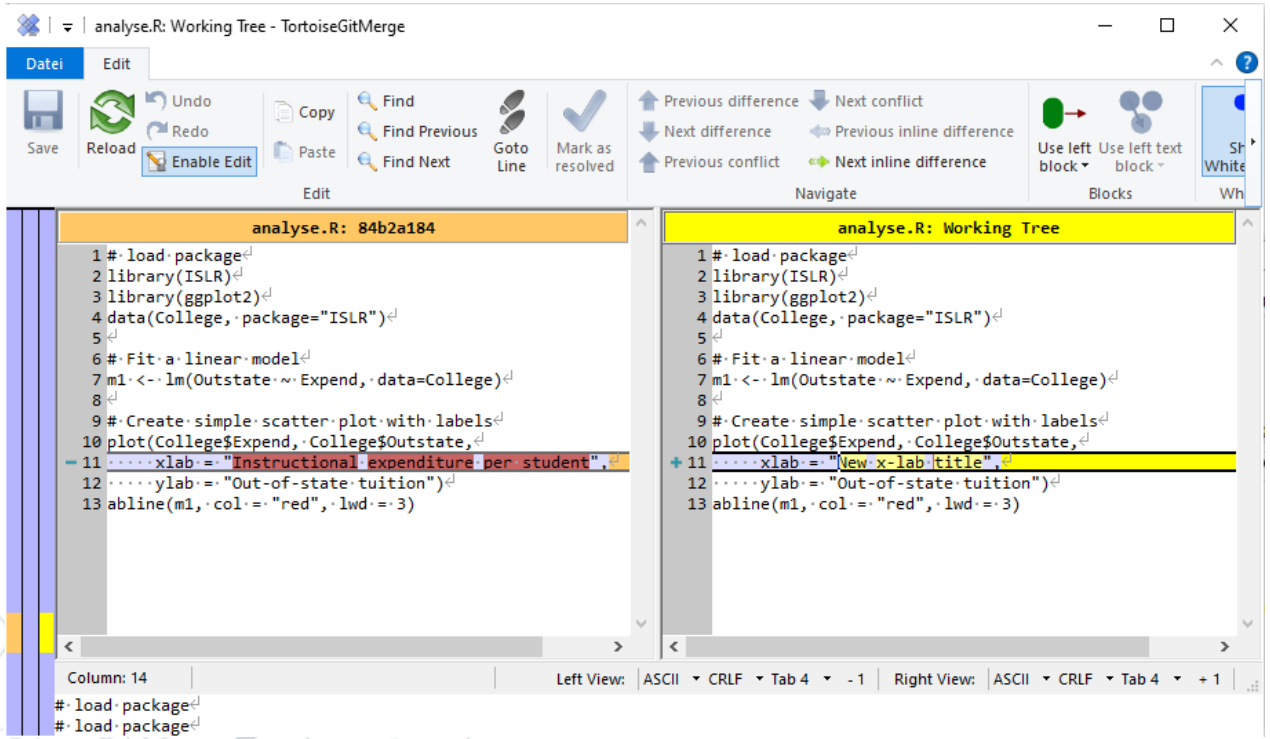
```
Admin@DESKTOP-79M1N5B MINGW64 /d/Privat/Präsentationen/Seminarreihe_2026/Vorträge/01_Git_Markdown_Jupyter/Git_init_Bsp (master)
$ git diff
diff --git a/analyse.R b/analyse.R
index cec153c..25807ab 100644
--- a/analyse.R
+++ b/analyse.R
@@ -8,6 +8,6 @@ m1 <- lm(Outstate ~ Expend, data=College)

 # Create simple scatter plot with labels
 plot(College$Expend, College$Outstate,
-      xlab = "Instructional expenditure per student",
+      xlab = "New x-lab title",
       ylab = "Out-of-state tuition")
 abline(m1, col = "red", lwd = 3)
\ No newline at end of file
```

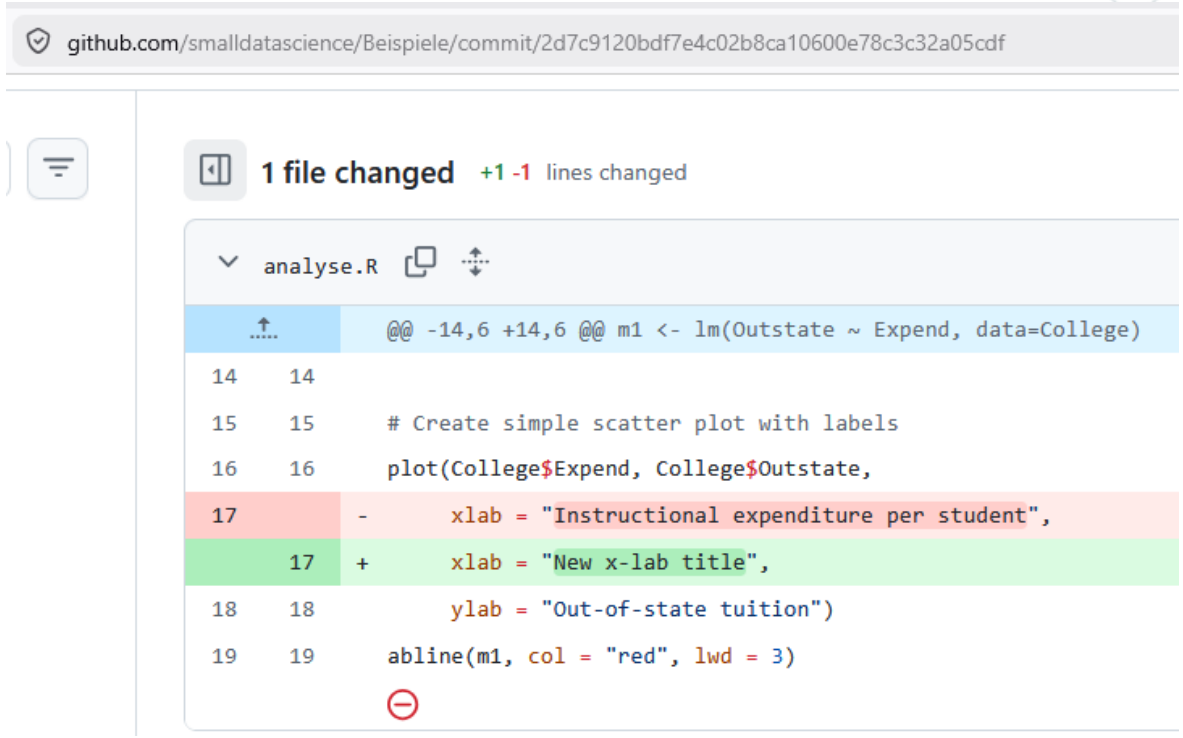
Welche Datei werden verwendet

Ausschnitt mit inhaltlichem Unterschied

Nächste Schritte | Nützliche Befehle – `git diff`

`git diff` Zeigt Unterschiede in Dateien





Nächste Schritte | Nützliche Befehle – `git status`

`git diff`	Zeigt Unterschiede in Dateien
`git status`	Zeigt den aktuellen Status des Repos

MINGW64:/d/Privat/Präsentationen/Seminarreihe_2026/Vorträge/01_Git_Markdown_Jupyter/Git_init_Bsp

```
Admin@DESKTOP-79M1N5B MINGW64 /d/Privat/Präsentationen/Seminarreihe_2026/Vorträge/01_Git_Markdown_Jupyter/Git_init_Bsp (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   analyse.R

no changes added to commit (use "git add" and/or "git commit -a")
```

Geänderte Datei **nicht** im staging Bereich

`git add analyse.R`

MINGW64:/d/Privat/Präsentationen/Seminarreihe_2026/Vorträge/01_Git_Markdown_Jupyter/Git_init_Bsp

```
Admin@DESKTOP-79M1N5B MINGW64 /d/Privat/Präsentationen/Seminarreihe_2026/Vorträge/01_Git_Markdown_Jupyter/Git_init_Bsp (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   analyse.R
```

Geänderte Datei im staging Bereich

Nächste Schritte | Nützliche Befehle – `git log`

`git diff`	Zeigt Unterschiede in Dateien
`git status`	Zeigt den aktuellen Status des Repos
`git log`	Zeigt die Historie eines Repos an

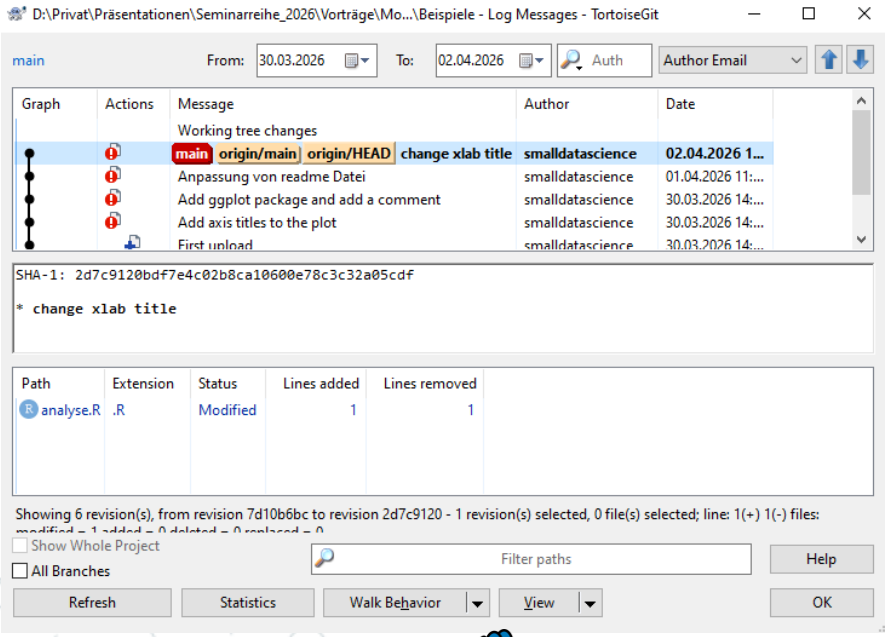
MINGW64:/d/Privat/Präsentationen/Seminarreihe_2026/Vorträge/01_Git_Markdown_Jupyter/Git_init_Bsp

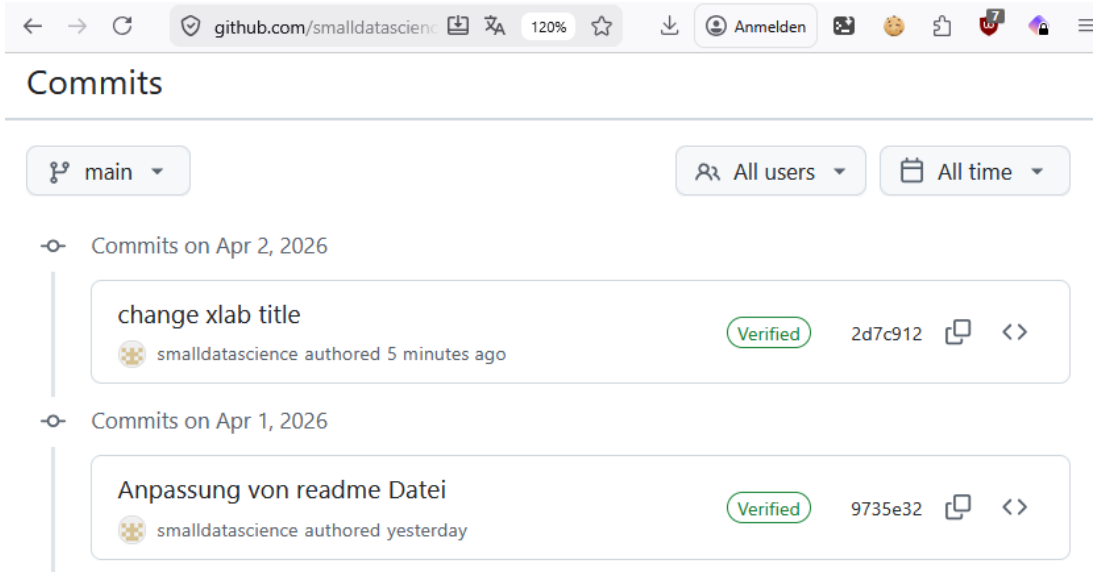
```
Admin@DESKTOP-79M1N5B MINGW64 /d/Privat/Präsentationen/Seminarreihe_2026/Vorträge/01_Git_Markdown_Jupyter/Git_init_Bsp (master)
$ git log
commit 84b2a184e4caa3ff17ccaee0712dd0561791bde0 (HEAD -> master)
Author: Fabian P <fabian.pribahsnik@gmx.at>
Date:   Wed Apr 1 10:33:54 2026 +0200

    Nicht notwenidge Code-Teile entfernen

commit c1d06755a9f177e48f6f96d41768767faa063456
Author: Fabian P <fabian.pribahsnik@gmx.at>
Date:   Wed Apr 1 09:43:02 2026 +0200

    Erster commit
```

1 Commit: Autor, Datum und Log-Nachricht

Nächste Schritte | Nützliche Befehle – `git log`

`git diff`	Zeigt Unterschiede in Dateien
`git status`	Zeigt den aktuellen Status des Repos
`git log`	Zeigt die Historie eines Repos an

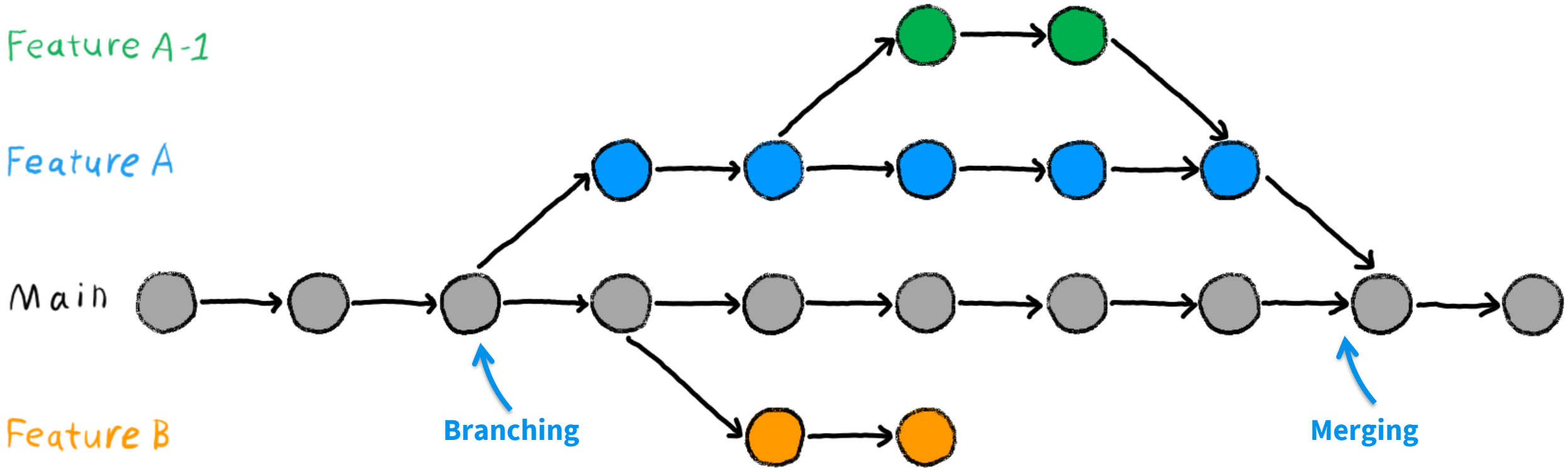




Nächste Schritte | Branching

- Main-Branch enthält als „single source of truth“ immer die Produktivversion des Projekts (getestet & produktiv eingesetzt)
- Oft ist es sinnvoll nicht im Hauptentwicklungsstrang zu arbeiten (besonders bei größeren/komplexeren Projekten):
  - Risikofreies experimentieren
  - Paralleles arbeiten
  - Neue Funktionen isoliert entwickeln

Nächste Schritte | Branching



Source: https://book.the-turing-way.org/reproducible-research/vcs/vcs-git-branches/

Referenzen

- Git:
  - Dokumentation: https://git-scm.com/docs
  - Buch: https://git-scm.com/book/en/v2
  - Cheat-Sheet: https://about.gitlab.com/images/press/git-cheat-sheet.pdf
  - …